Lecture 05

Object-Oriented Programming

Constructor, File Processing

Prepared by **Hok Tin**

Content

- Super keyword
- Constructor
- File Processing

Super keyword

- The **super keyword** refers to superclass (**parent**) objects

```
class MotoBike {
    private int wheelNum = 2;
    void wheel() {
        System.out.println("There are " + wheelNum + " wheels.");
    }
}

class Tuktuk extends MotoBike {
    private int wheelNum = 3;
    public void myWheel() {
        System.out.println("There are " + wheelNum + " wheels.");
    }
    public void parentWheel() {
        super.wheel();
    }
}
```

```
public class Demo {
    public static void main(String[] args) {
        Tuktuk tt = new Tuktuk();

        tt.myWheel();
        tt.parentWheel();
    }
}
```

Output:

```
There are 3 wheels.
There are 2 wheels.
```

Super keyword

```
class Math {
    double pi() {
        return 3.14;
    }
}

class AdvancedMath extends Math {
    public double circleSurface(int radius) {
        return super.pi() * (radius * radius);
    }
}
```

```
public class Demo6 {
    public static void main(String[] args) {
        AdvancedMath am = new AdvancedMath();
        System.out.print("Circle radius is: " + am.circleSurface(50));
    }
}
```

Output:

```
Circle radius is: 7850.0
```

Constructor

Constructor in Java is called when an instance of the class is created

✍ Terms used in Constructor

- The constructor's name is always the same as the class name
- A Constructor must have no explicit return type
- A Java constructor cannot be abstract, static, final, and synchronized

```
class Bike {
  Bike() {
      System.out.println("Bike object is created~");
  }

  public static void main(String args[]) {
      Bike b = new Bike();
  }
}
```

Constructor

Constructor to initial state for an object when the object is created

```java
class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void display() {
        System.out.println(id + " " + name);
    }
}

public class Demo7 {
    public static void main(String args[]) {
        Student s1 = new Student(1111, "Tola");
        Student s2 = new Student(2222, "Kompheak");

        s1.display();
        s2.display();
    }
}
```

Vs

```java
class Student {
    int id;
    String name;

    void insertData(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void display() {
        System.out.println(id + " " + name);
    }
}

public class Demo {
    public static void main(String args[]) {
        Student s1 = new Student();
        s1.insertData(111, "Tola");

        Student s2 = new Student();
        s2.insertData(222, "Kompheak");

        s1.display();
        s2.display();
    }
}
```

Processing files

File Writer

- The **PrintWriter** class offers the functionality to write to files.
- The *constructor* of the **PrintWriter** class receives as its parameter a string that represents the location of the target file
- Adding ***throws Exception*** to the method definition

```
1  public class FileTest {
2      public static void main(String []args) throws Exception{
3          PrintWriter writer = new PrintWriter("file.txt");
4          writer.println("Hello file!"); //writes the string "Hello file!" and line change to the file
5          writer.println("More text");
6          writer.print("And a little extra"); // writes the string "And a little extra" to the file without a line change
7          writer.close(); //closes the file and ensures that the written text is saved to the file
8      }
9  }
```

Processing files

File Reader

- The **Scanner** class offers the functionality to read line by line text from file, which created from **File** class.
- The *constructor* of the **Scanner** class receives as its parameter as a File object
- Adding ***throws Exception*** to the method definition

```
1  public class FileTest {
2      public static void main(String []args) throws Exception {
3
4          Scanner scanner = new Scanner(new File("fileName.txt"));
5          while (scanner.hasNextLine()) {
6              // process the line
7              String line = scanner.nextLine();
8              System.out.println(line);
9          }
10
11     }
12 }
```

- Check if a line is blank or empty

```
while (scanner.hasNextLine()) {
    String line = scanner.nextLine();

    // if the line is blank we do nothing
    if (line.isEmpty()) {
        continue;
    }

    // do something with the data
```

Good luck 👌

😉 **References:**

https://www.javatpoint.com/java-constructor

https://java-programming.mooc.fi/part-4/1-introduction-to-object-oriented-programming

https://www.javatpoint.com/super-keyword